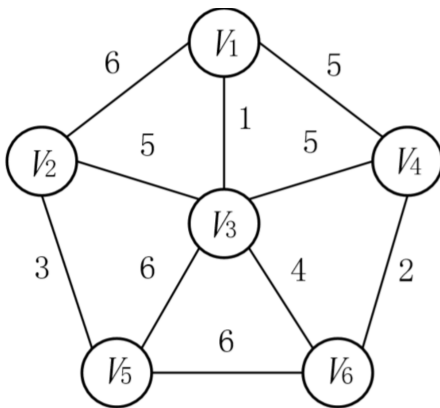


1. Prime

算法简单描述

- 1).输入：一个加权连通图，其中顶点集合为 V ，边集合为 E ；
- 2).初始化： $V_{new} = \{x\}$ ，其中 x 为集合 V 中的任一节点（起始点）， $E_{new} = \{\}$ ，为空；
- 3).重复下列操作，直到 $V_{new} = V$ ：
 - a.在集合 E 中选取权值最小的边 $\langle u, v \rangle$ ，其中 u 为集合 V 中的元素，而 v 不在 V_{new} 集合当中，并且 $v \in V$ （如果存在有多条满足前述条件即具有相同权值的边，则可任意选取其中之一）；
 - b.将 v 加入集合 V_{new} 中，将 $\langle u, v \rangle$ 边加入集合 E_{new} 中；
- 4).输出：使用集合 V_{new} 和 E_{new} 来描述所得到的最小生成树。

图例

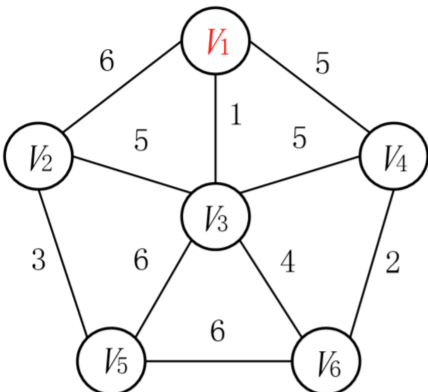


已选的点集 $V_{new} = \{ \}$

已选的边集 $E_{new} = \{ \}$

当前可选点集 $U = \{ \}$

当前所选边的权值总和：0

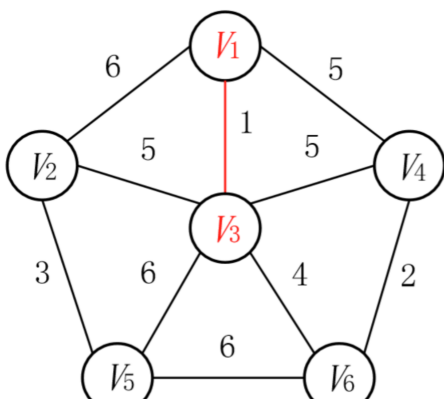


已选的点集 $V_{new} = \{ V_1 \}$

已选的边集 $E_{new} = \{ \}$

当前可选点集 $U = \{ V_2, V_3, V_4 \}$

当前所选边的权值总和：0

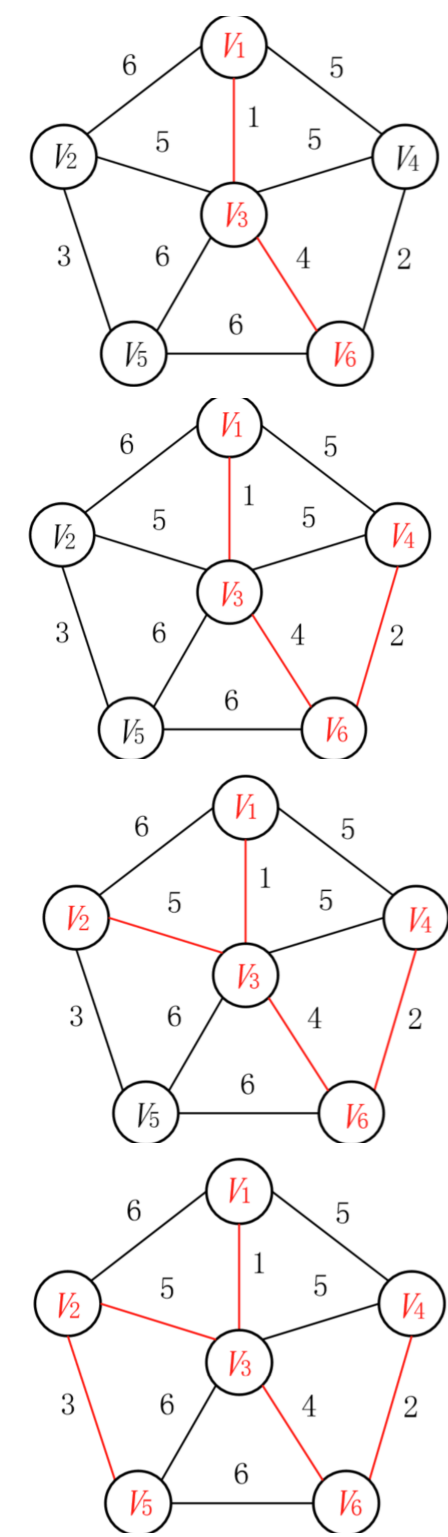


已选的点集 $V_{new} = \{ V_1, V_3 \}$

已选的边集 $E_{new} = \{ \langle V_1, V_3 \rangle \}$

当前可选点集 $U = \{ V_2, V_4, V_5, V_6 \}$

当前所选边的权值总和：1



代码实现

C++ 简易实现

- visit[]:表示哪些点已经加入到树中
- mp[]: 邻接矩阵存图
- path[]: 例如 · path[5]表示最终的最小生成树中到5这个点的前驱；相当于记录了最小生成树的边

```
int mp[2005][2005]; // 邻接矩阵存图
bool vis[2005];
```

```

int path[2005]; //由那条边
int dis[2005];

int prime(){
    int minn, u = 1, sum = 0;
    //初始化
    memset(vis, false, sizeof(vis));
    for(int i = 1; i <= n; i++){
        dis[i] = mp[1][i];
    }
    vis[1] = true;
    for(int i = 2; i <= n; i++){
        minn = inf;
        for(int j = 1; j <= n; j++){
            if(!vis[j] && minn > dis[j]){
                u = j;
                minn = dis[j];
            }
        }
        vis[u] = true;
        for(int j = 1; j <= n; j++){
            if(!vis[j] && dis[j] > mp[u][j]){
                dis[j] = mp[u][j];
                path[j] = u;
            }
        }
    }
    for(int i = 2; i <= n; i++){
        sum += dis[i];
    }
    return sum;
}

```

Python实现：

```

"""
Prim类接受两个参数n,m，分别表示录入的点和边的个数（点的编号从1开始递增）
接下来是m行，每行3个数x,y,length，表示点x和点y之间存在一条长度为length的边
程序最终会打印出该无向图中的最小生成树的各个边和最小权值
"""

class Node:
    """
    Node类存放节点信息，
    node是与某个节点（实际上是用列表索引值代表的）相连接的点
    length表示这两个点之间的权值
    """
    def __init__(self, node, length):
        self.node = node
        self.length = length

class Edge:

```

```

"""
Edge类表示边的信息
x,y表示这条边的两个顶点
length为<x,y>之间的距离
"""
def __init__(self, x, y, length):
    self.x = x
    self.y = y
    self.length = length

class Prim:
    """Prim算法：求解无向连通图中的最小生成树 """
    def __init__(self, n, m):
        self.n = n # 输入的点个数
        self.m = m # 输入的边个数
        self.v = [[] for i in range(n+1)] # 存放所有节点之间的可达关系与距离
        """图的邻接表的表示法，v是一个二维列表，其索引值代表当前节点，索引值对应的一维列
表中存放的
是与以当前索引值为顶点的相连的节点信息"""
        self.e = [] # 存放与当前已选节点相连的边
        """e是一个一维列表，存放的是与当前顶点相连的所有的边的信息"""
        self.s = [] # 存放最小生成树里的所有边
        self.vis = [False for i in range(n+1)] # 标记每个点是否被访问过，False未访
问

    def graphy(self):
        """构建图，这里用的是邻接表"""
        for i in range(self.m):
            x, y, length = list(map(int, input().split()))
            self.v[x].append(Node(y, length)) # 与x相连的y节点记录在二维列表v的x索引
值对应的一维列表中
            self.v[y].append(Node(x, length)) # 与y相连的x节点记录在二维列表v的y索引
值对应的一维列表中
            # print(self.v)

    def insert(self, point):
        """往Vnew中插入一个新的点"""
        for i in range(len(self.v[point])):
            """把与point节点相连的且未被访问的节点的边加入列表e中"""
            if not self.vis[self.v[point][i].node]:
                self.e.append(Edge(point, self.v[point][i].node, self.v[point]
[i].length))
            self.vis[point] = True
            self.e = sorted(self.e, key=lambda e: e.length) # 把e中的所有边按边的长度从
小到大排序
            # for i in self.e:
            #     print(i.length)

    def run(self, start):
        """执行函数：求解录入的无向连通图的最小生成树"""
        self.insert(start) # start为选择的开始顶点
        while self.n - len(self.s) > 1: # 最小生成树的边数=图中节点数-1，因此可以利
用这个性质来作为循环条件

```

```

        for i in range(len(self.e)): # 按序遍历所有边
            if not self.vis[self.e[i].y]: # 如果待检测的第二个位置上的点未访问
                过，则说明这条边满足一端在Vnew中，另一端不在
                self.s.append(self.e[i]) # 则需要将该边放进结果集合中，因为第一个遇到的肯定是权值最小的，在插入函数中已经排好序了
                self.insert(self.e[i].y) # 同时将该边的另一个节点插入Vnew中
                break # 找到一条符号条件的便后就退出for循环

def print(self):
    """输出信息"""
    print(f'当前录入总边数为：{len(self.e)}\n其中构成最小生成树的边为：')
    edge_sum = 0
    for i in range(len(self.s)):
        print(f'边<{self.s[i].x},{self.s[i].y}> = {self.s[i].length}')
        edge_sum += self.s[i].length
    print(f'最小生成树的权值为：{edge_sum}')

def main():
    n, m = list(map(int, input().split()))
    prim = Prim(n, m)
    prim.graphy()
    prim.run(1)
    prim.print()

if __name__ == '__main__':
    main()

```

```
6 10
```

```
1 2 6
```

```
1 3 1
```

```
1 4 5
```

```
2 3 5
```

```
2 5 3
```

```
3 4 5
```

```
3 5 6
```

```
3 6 4
```

```
4 6 2
```

```
5 6 6
```

输入：

```
当前录入总边数为：10
其中构成最小生成树的边为：
边<1,3> = 1
边<3,6> = 4
边<6,4> = 2
边<3,2> = 5
边<2,5> = 3
最小生成树的权值为：15
```

输出：

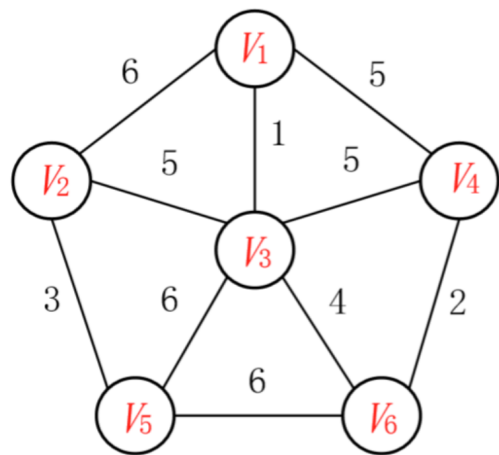
2.Kruskal

算法简单描述：

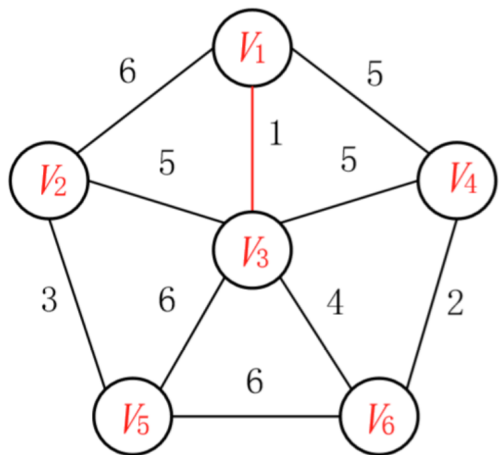
- 1. 设无向连通图Graph有v个顶点，e条边；
- 2. 新建图Graphnew，Graphnew拥有与原图中相同的v个顶点，但没有边；
- 3. 将原图Graph中所有边按权值从小到大排序；
- 4. 进入一层循环，该循环从权值最小的边开始遍历每条边，直至图Graphnew中所有的节点都在同一个连通分量中。循环体内的内容如下：if（这条边连接的两个节点于图Graphnew中不在同一个连通分量中）则添加这条边到图Graphnew中；

图例

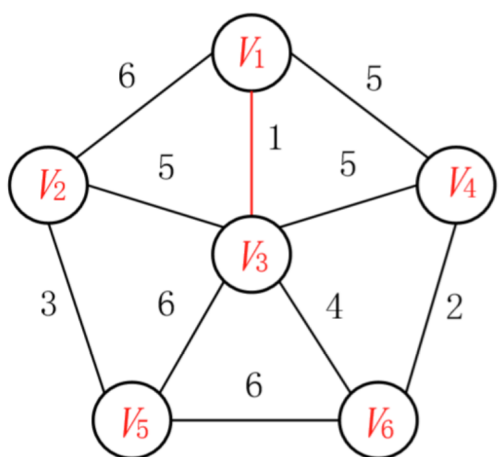
序号	边	长度
1	$\langle V_1, V_3 \rangle$	1
2	$\langle V_4, V_6 \rangle$	2
3	$\langle V_2, V_5 \rangle$	3
4	$\langle V_3, V_6 \rangle$	4
5	$\langle V_1, V_4 \rangle$	5
6	$\langle V_2, V_3 \rangle$	5
7	$\langle V_3, V_4 \rangle$	5
8	$\langle V_1, V_2 \rangle$	6
9	$\langle V_3, V_5 \rangle$	6
10	$\langle V_5, V_6 \rangle$	6



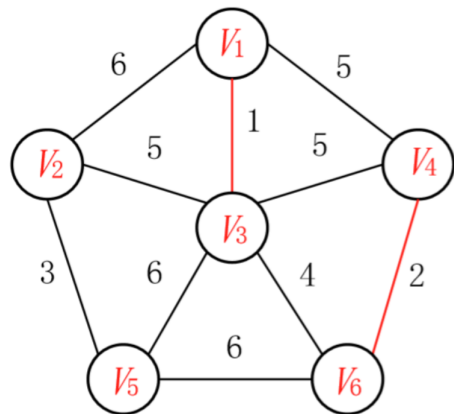
已选边集 $E = \{ \}$



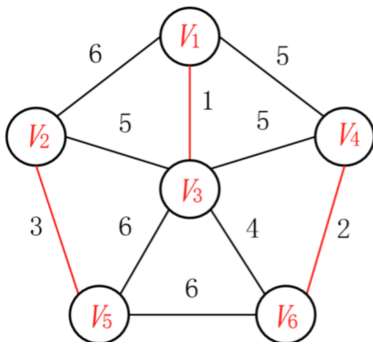
已选边集 $E = \{ \langle V_1, V_3 \rangle \}$



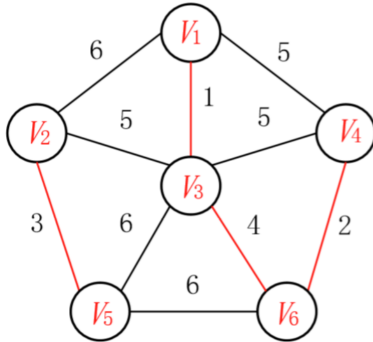
已选边集 $E = \{ \langle V_1, V_3 \rangle \}$



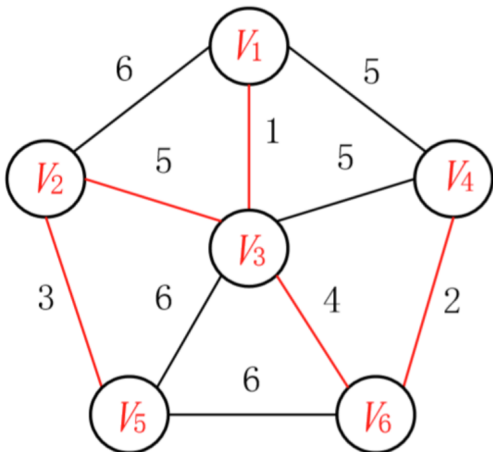
已选边集 $E = \{ \langle V_1, V_3 \rangle, \langle V_4, V_6 \rangle \}$



已选边集 $E = \{ \langle V_1, V_3 \rangle, \langle V_4, V_6 \rangle, \langle V_2, V_5 \rangle \}$



已选边集 $E = \{ \langle V_1, V_3 \rangle, \langle V_4, V_6 \rangle, \langle V_2, V_5 \rangle, \langle V_3, V_6 \rangle \}$



已选边集 $E = \{ \langle V_1, V_3 \rangle, \langle V_4, V_6 \rangle, \langle V_2, V_5 \rangle, \langle V_3, V_6 \rangle, \langle V_2, V_3 \rangle \}$

代码实现

C++:

题目链接：[1789 -- Truck History \(poj.org\)](https://poj.org/problem?id=1789)

```
/**
time : 2021/08/31
author: 小风时雨摘运霞
**/
#include<iostream>
#include<cstring>
#include<algorithm>
#define MAX 2002000
using namespace std;

int parent[2002], n;
char str[2002][10];

struct node{
```



```

    int s, e, v;
}g[MAX];
int distance(char a[], char b[]){
    int count = 0;
    for(int i = 0; i < 7; i++){
        if(a[i] != b[i]) count ++;
    }
    return count;
}
// 排序边
bool cmp(node a, node b){
    return a.v < b.v;
}

// 查找根先节点
int find(int x){
    return x == parent[x] ? x : parent[x] = find(parent[x]);
}
// 并查集合并
bool union_(int a, int b){
    int fa = find(a);
    int fb = find(b);
    if(fa != fb){
        parent[fa] = fb;
        return true;
    }
    return false;
}
int main(){
    int temp;
    while(cin >> n && n){
        int k = 0, sum = 0;
        for(int i = 0; i < n; i++){
            scanf("%s", str[i]);
        }
        for(int i = 0; i < n - 1; i++){
            for(int j = i + 1; j < n; j++){
                temp = distance(str[i], str[j]);
                g[k].s = i;
                g[k].e = j;
                g[k].v = temp;
                k ++; // 统计多少条
            }
        }
        sort(g, g + k, cmp); //排序
        for(int i = 0; i <= n; i++){
            parent[i] = i; //建立初始树
        }
        for(int i = 0; i < k; i++){
            if(union_(g[i].s, g[i].e)){ //查询两个节点是否有共同的根，没有就加入，有就
说明在同一颗树
                sum += g[i].v;
            }
        }
    }
}

```

```
        printf("The highest possible quality is 1/%d.\n", sum);
    }
    return 0;
}
```

3.Dijkstra

算法简单描述

Dijkstra(迪杰斯特拉)算法是典型的单源最短路径算法，用于计算一个节点到其他所有节点的最短路径。主要特点是以起始点为中心向外层层扩展，直到扩展到终点为止。Dijkstra算法是很有代表性的最短路径算法，在很多专业课程中都作为基本内容有详细的介绍，如数据结构，图论，运筹学等等。注意该算法要求图中不存在**负权边**。

问题描述：在无向图 $G=(V,E)$ 中，假设每条边 $E[i]$ 的长度为 $w[i]$ ，找到由顶点 V_0 到其余各点的最短路径。（单源最短路径）

1)算法思想：设 $G=(V,E)$ 是一个带权有向图，把图中顶点集合 V 分成两组，第一组为已求出最短路径的顶点集合（用 S 表示，初始时 S 中只有一个源点，以后每求得一条最短路径，就将加入到集合 S 中，直到全部顶点都加入到 S 中，算法就结束了），第二组为其余未确定最短路径的顶点集合（用 U 表示），按最短路径长度的递增次序依次把第二组的顶点加入 S 中。在加入的过程中，总保持从源点 v 到 S 中各顶点的最短路径长度不大于从源点 v 到 U 中任何顶点的最短路径长度。此外，每个顶点对应一个距离， S 中的顶点的距离就是从 v 到此顶点的最短路径长度， U 中的顶点的距离，是从 v 到此顶点只包括 S 中的顶点为中间顶点的当前最短路径长度。

2)算法步骤：

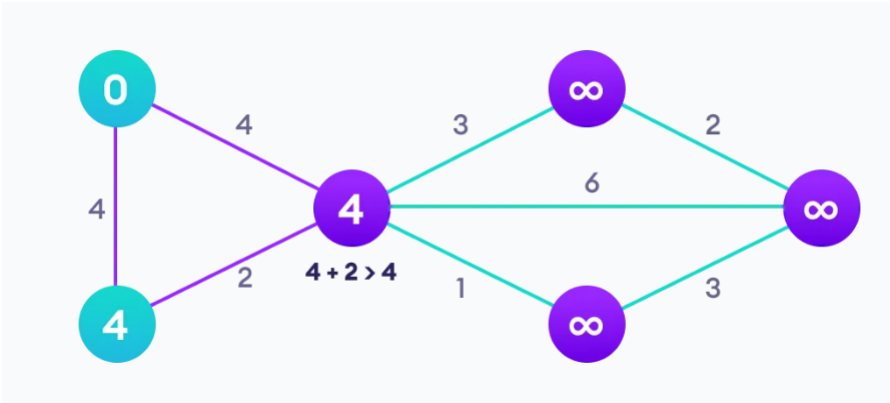
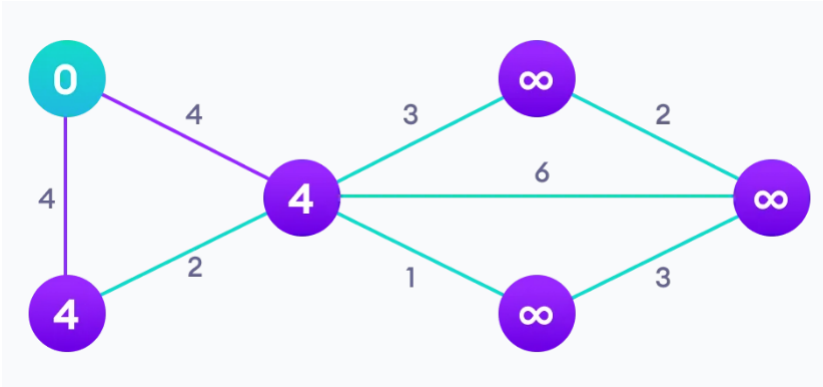
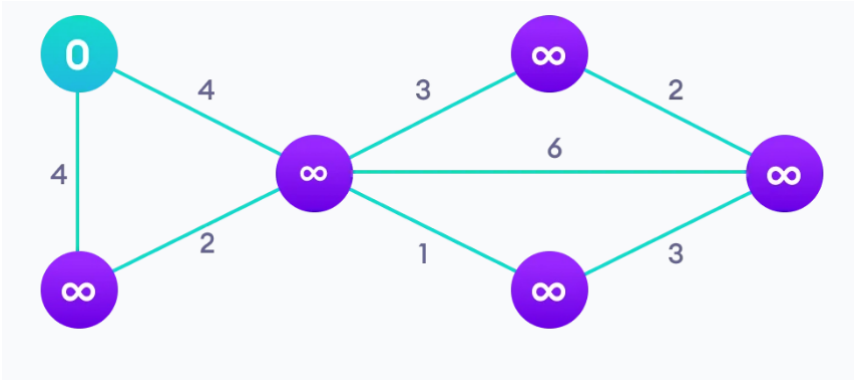
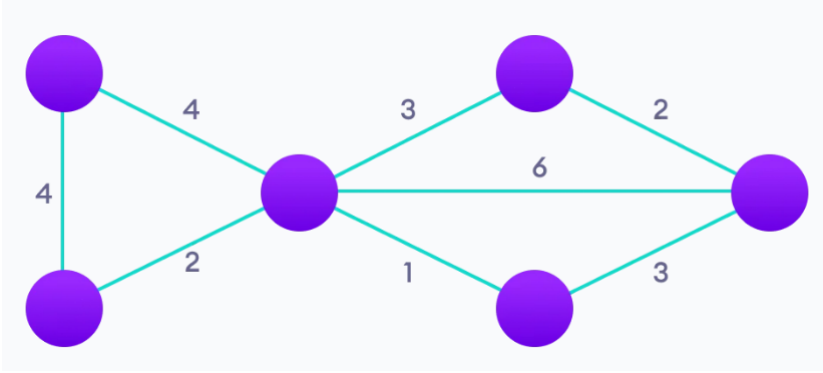
a.初始时， S 只包含源点，即 $S = \{v\}$ ， v 的距离为0。 U 包含除 v 外的其他顶点，即： $U = \{\text{其余顶点}\}$ ，若 v 与 U 中顶点 u 有边，则 $\langle u,v \rangle$ 正常有权值，若 u 不是 v 的出边邻接点，则 $\langle u,v \rangle$ 权值为 ∞ 。

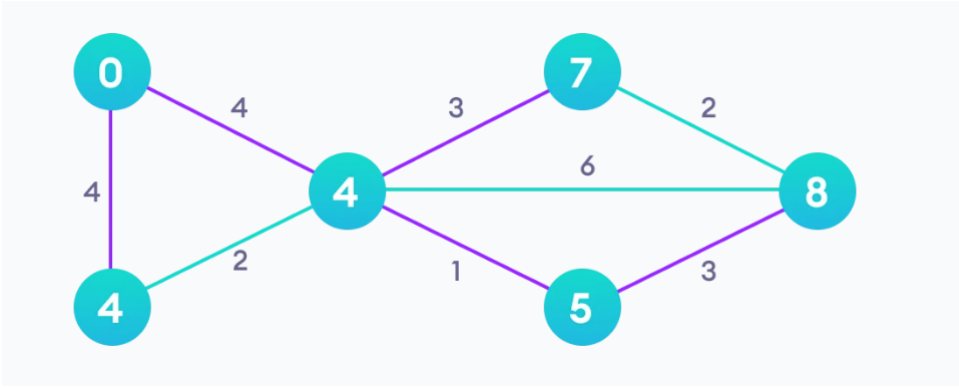
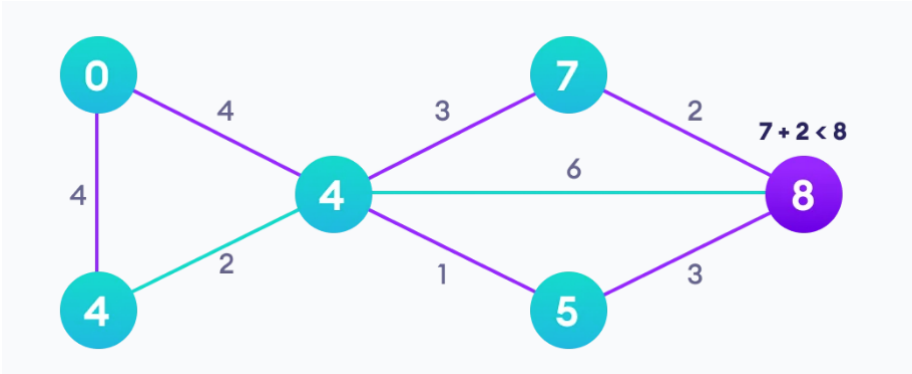
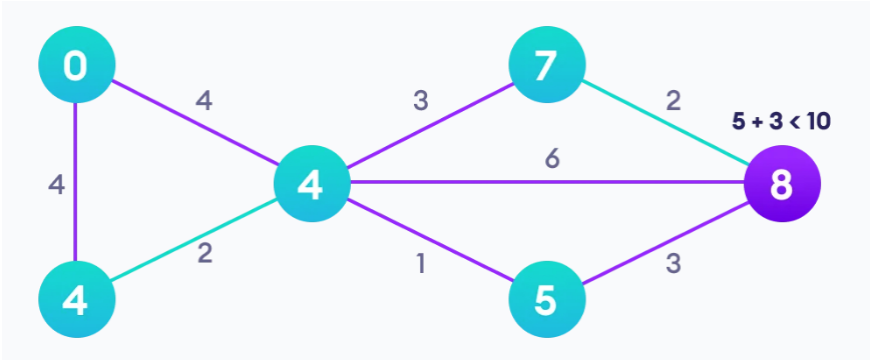
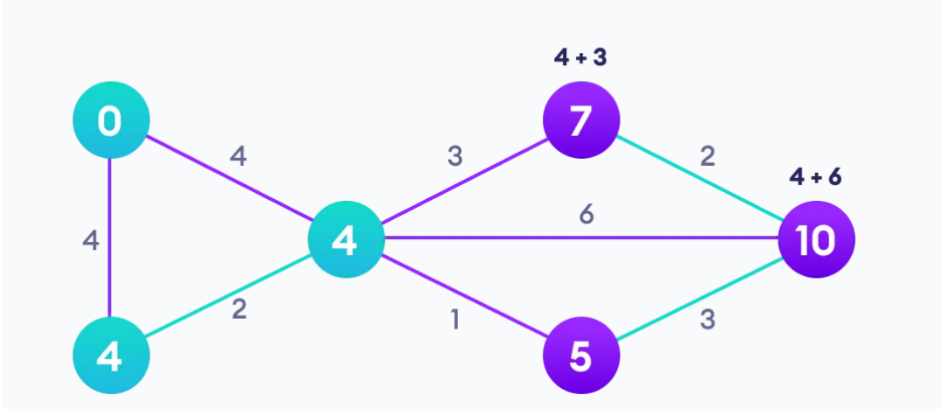
b.从 U 中选取一个距离 v 最小的顶点 k ，把 k ，加入 S 中（该选定的距离就是 v 到 k 的最短路径长度）。

c.以 k 为新考虑的中间点，修改 U 中各顶点的距离；若从源点 v 到顶点 u 的距离（经过顶点 k ）比原来距离（不经过顶点 k ）短，则修改顶点 u 的距离值，修改后的距离值的顶点 k 的距离加上边上的权。

d.重复步骤b和c直到所有顶点都包含在 S 中。

图例





代码实现

C++

- trackPath数组：记录u到其他顶点的最短路径
- path数组：记录u到其他顶点的最短路径
- rec数组：记录那些点已经计算了，放到了集合V中

```

void dijkstra1(int u){
    int inf = 0x3f3f3f3f; // 无穷大
    memset(trackPath, -1, sizeof(trackPath)); // 记录u到其他顶点的最短路径
    memset(path, inf, sizeof(path)); // 记录u到其他顶点的最短路径
    memset(rec, false, sizeof(rec)); // 记录那些点已经计算了，放到了集合V中
    path[u] = 0; // 自己到自己距离为0
    int m;
    for(int i = 0; i <= n; i++){
        for(int j = 1, m = inf; j <= n; j++){
            if(!rec[j] && path[j] < m){
                m = path[j];
                u = j;
            }
        }
        rec[u] = true;
        for(int k = 1; k <= n; k++){
            if(!rec[k]){
                // 松弛操作
                path[k] = min(path[k], path[u] + g[u][k]);
                trackPath[k] = u; // 记录最短路径u到k, k的前驱：可以追踪最短路径的线路
            }
        }
    }
}

```

堆优化

堆优化的主要思想就是使用一个优先队列（就是每次弹出的元素一定是整个队列中最小的元素）来代替最近距离的查找，用邻接表代替邻接矩阵，这样可以大幅度节约时间开销，就是对下面的代码进行优化，使用优先队列，就不用每次 $O(n)$ 查找，效率 $O(n\log n)$ 。

```

for(int j = 1, m = inf; j <= n; j++){
    if(!rec[j] && path[j] < m){
        m = path[j];
        u = j;
    }
}

```

```

struct Node{
    int u, dis;
    Node(int u, int dis): u(u), dis(dis){}
    bool operator < (const Node& t) const{ // 重小到排序
        return dis > t.dis;
    }
}

void dijkstra2(int u){
    memset(path, inf, sizeof(path));
    memset(rec, false, sizeof(rec));
}

```

```

memset(trackPath, -1, sizeof(trackPath)); //记录u到其他顶点的最短路径
path[u] = 0;
priority_queue<Node>q;
q.push(Node(u, 0)); //将初始点放入队列
while(q.empty()){
    int uu = q.top().u;
    q.pop(); //logn
    if(rec[uu]) continue;
    rec[uu] = true;
    for(int k = 1; k <= n; k++){
        if(!rec[k] && path[k] > path[uu] + g[uu][k]){
            path[k] = path[uu] + g[uu][k];
            q.push(Node(k, path[k])); // 将其放入优先队列，后面直接弹出元素就是最
            trackPath[k] = uu;
        }
    }
}
}
}

```

近的

Fleury

除非万不得已，否则不走割边